# Ders 5 Çalışma Özeti

## Ders 5 Çalışma Özeti

### Genel Konular

- Dersin tarihi 21 Ekim 2020 Çarşamba. Ders Zoom üzerinden yapıldı. Hoca dönemin üçüncü haftası olduğunu söylüyor.
  - Ders "2 saat konu + 1 saat uygulama" düzeniyle yapılıyor. Bu kayıt konu anlatımı kısmı.
  - Başta birkaç dakika ses sorunu var. Asıl anlatım yaklaşık 00:07:50'de başlar.
- Matematiksel analizde kullanılan formüller
  - Logaritma kuralları.
  - Kombinatorik: permütasyon ($n!$), kombinasyon ($C(n,k)$), alt küme sayısı ($2^n$).
  - Toplam formülleri (summation formulas) ve toplam üzerinde işlem kuralları (sum manipulation rules).
  - Alta yuvarlama (floor) ve üste yuvarlama (ceiling).
- Özyinelemesiz algoritmaların matematiksel analizi (mathematical analysis of non-recursive algorithms)
  - Örnek 1: dizideki en büyük elemanı bulma (MaxElement).
  - Örnek 2: dizideki elemanların hepsi farklı mı (UniqueElements).
  - Çalışma süresi tahmini: $T(n) \approx c_{op} C(n)$.
- Özyinelemeli algoritmaların matematiksel analizi (mathematical analysis of recursive algorithms)
  - Örnek 1: faktöriyel. Rekürans bağıntısı (recurrence relation), başlangıç koşulu (initial condition), geriye doğru yerine koyma (backward substitution).
  - Örnek 2: bir sayının ikilik gösterimindeki basamak sayısı (BinRec).

### Hocanın Özellikle Vurguladığı Kısımlar

- Analizin ilk iki adımı: girdi boyutunu ($n$) ve temel işlemi (basic operation) belirlemek.
  - Temel işlem atama, aritmetik işlem ya da karşılaştırma olabilir.
  - Temel işlem en içteki döngüde (innermost loop) aranır. Çünkü en çok tekrar eden iş odur.
- Her iç içe döngü bir toplam ($\sum$) işaretine karşılık gelir.
  - Arka arkaya iki ayrı döngü iç içe değildir. Karmaşıklığı artırmaz.
- Her durum (en kötü, ortalama, en iyi) aynı karmaşıklığa sahipse genel $C(n)$ yazılır. Değilse $C_{worst}$, $C_{avg}$, $C_{best}$ ayrı yazılır.
  - Aksi belirtilmedikçe en kötü durum (worst case) hesaplanır.
- Bu derste amaç, sonucu tahmin etmek değil, matematiksel analizi adım adım göstermektir.
  - Sınavda da adım adım çözüm beklenir.
- Özyinelemeli algoritmada temel işlem, genellikle fonksiyonun kendini çağırdığı satırdaki işlemdir.
  - Durma koşulundaki karşılaştırma temel işlem olarak alınmaz. O satır başlangıç koşulunu verir.
  - Fonksiyon çağrısının kendisi temel işlem olarak sayılmaz.
- Faktöriyelde $n = 0$ durumu gözden kaçırılmamalı.
  - Durma koşulu `n == 1` olursa `F(0)` çağrısı sonsuz özyinelemeye girer. Doğru durma koşulu `n == 0` iken `1` döndürmektir.

- Özyinelemeli çözüm daha kısa koddur ama anlaşılması daha zordur. Her çağrının bellek ve zaman ek yükü (overhead) vardır.
- Derse canlı katılım:
  - Hoca öğrencilere adıyla soru soruyor. Cevap vermeyen çok öğrenci olduğu için ileride önlem alacağını ve puan kırabileceğini söyledi.
  - Kayıttan izlemek, dersin eş zamanlı yapılmasının yerini tutmaz.

## Kısa Tekrar Notları

- Toplam formülleri:
  - $\sum_{i=l}^{u} 1 = u - l + 1$ ($l \leq u$).
  - $\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \approx \frac{1}{2}n^2$.
  - $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{1}{3}n^3$.
  - $\sum_{i=1}^{n} i^k \approx \frac{1}{k+1}n^{k+1}$.
  - $\sum_{i=0}^{n} a^i = \frac{a^{n+1}-1}{a-1}$ ($a \neq 1$). Özel durum: $\sum_{i=0}^{n} 2^i = 2^{n+1} - 1$.
  - $\sum_{i=1}^{n} i\,2^i = (n-1)2^{n+1} + 2$.
  - $\sum_{i=1}^{n} \frac{1}{i} \approx \ln n + \gamma$, $\gamma \approx 0{,}5772$.
- Toplam işlem kuralları:
  - $\sum c\,a_i = c \sum a_i$.
  - $\sum (a_i \pm b_i) = \sum a_i \pm \sum b_i$.
  - $\sum_{i=l}^{u} a_i = \sum_{i=l}^{m} a_i + \sum_{i=m+1}^{u} a_i$ ($l \leq m < u$).
- Yuvarlama: $\lceil 2{,}7 \rceil = 3$, $\lceil -2{,}7 \rceil = -2$, $\lfloor 2{,}7 \rfloor = 2$, $\lfloor -2{,}7 \rfloor = -3$.
- MaxElement: temel işlem karşılaştırma, $C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n)$.
- UniqueElements (en kötü durum): $C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \frac{(n-1)n}{2} \in \Theta(n^2)$.
- Faktöriyel: $M(n) = M(n-1) + 1$, $M(0) = 0$. Sonuç $M(n) = n$.
- BinRec: $A(n) = A(\lfloor n/2 \rfloor) + 1$, $A(1) = 0$. $n = 2^k$ için sonuç $A(n) = \log_2 n \in \Theta(\log n)$.

## Detaylı Açıklamalar

### Özyinelemesiz algoritmaların analizi

Genel adımlar:

1. Girdi boyutunu gösteren parametreyi belirle (çoğunlukla $n$).
2. Temel işlemi belirle (en içteki döngüde).
3. Temel işlemin sayısı yalnız $n$'ye mi bağlı, yoksa girdinin dizilişine de mi bağlı? Bağlıysa en kötü, ortalama ve en iyi durum ayrı incelenir.
4. Temel işlemin kaç kez yapıldığını bir toplam olarak yaz.
5. Toplamı formüllerle kapalı biçime getir ve büyüme derecesini bul.

#### MaxElement

```
MaxElement(A[0..n-1])
  maxval <- A[0]
  for i <- 1 to n-1 do
    if A[i] > maxval
      maxval <- A[i]
  return maxval
```

- İki aday temel işlem var: `A[i] > maxval` karşılaştırması ve `maxval <- A[i]` ataması.
  - Karşılaştırma her turda yapılır. Atama yalnız karşılaştırma doğruysa yapılır. Bu yüzden temel işlem karşılaştırmadır.
- Karşılaştırma sayısı dizinin dizilişine bağlı değildir. Üç durum aynıdır: $C(n) = n - 1 \in \Theta(n)$.
- Hoca aynı algoritmayı akış diyagramıyla da gösterdi. Sözde kod (pseudocode) daha hızlı yazıldığı için derste genelde sözde kod kullanılacak.

### UniqueElements

```
UniqueElements(A[0..n-1])
  for i <- 0 to n-2 do
    for j <- i+1 to n-1 do
      if A[i] = A[j] return false
  return true
```

- Döngü sayısı belli olmadığı halde `for` kullanılabildi. Çünkü fonksiyondan `return` ile çıkılıyor.
  - Hoca normalde döngüden `break` ile çıkmayı önermiyor. Fonksiyon içinde `return` ile çıkış ise kabul edilebilir.
- En iyi durum: ilk karşılaştırılan iki eleman eşit. Bu yüzden durumlar farklıdır ve en kötü durum hesaplanır.
- En kötü durum, eşit eleman olmadığı ya da son iki elemanın eşit olduğu durumdur:

$$C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)n}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2)$$

### Çalışma süresi tahmini

- $T(n) \approx c_{op}\, C(n)$.
- $c_{op}$, temel işlemin bir kez yapılma süresidir. Örneğin karşılaştırma için $c_c$, çarpma için $c_m$.
- Diğer işlemler de hesaba katılabilir. Ama analizi genellemek için yalnız temel işlem sayılır.

### Özyinelemeli algoritmaların analizi: faktöriyel

```
F(n)
  if n = 0 return 1
  else return F(n-1) * n
```

- Temel işlem çarpmadır. Sayısı $M(n)$ ile gösterilir.
- Rekürans bağıntısı: $M(n) = M(n-1) + 1$, $n > 0$.
  - $M(n-1)$: $F(n-1)$'i hesaplamak için yapılan çarpmalar.
  - $+1$: $F(n-1)$'i $n$ ile çarpmak.
- Başlangıç koşulu: $n = 0$ iken çarpma yapılmaz. $M(0) = 0$.
- Geriye doğru yerine koyma:

$$M(n) = M(n-1) + 1 = M(n-2) + 2 = M(n-3) + 3 = \cdots = M(n-i) + i$$

- $i = n$ alınır: $M(n) = M(0) + n = n$.

### İkilik basamak sayısı: BinRec

```
BinRec(n)            // n pozitif tam sayı
  if n = 1 return 1
  else return BinRec(floor(n/2)) + 1
```

- Sonuç, $n$'nin ikilik gösterimindeki basamak sayısıdır.
- Temel işlem, özyinelemeli çağrıdan sonraki toplamadır. Sayısı $A(n)$.
- $A(n) = A(\lfloor n/2 \rfloor) + 1$, $n > 1$. Başlangıç koşulu $A(1) = 0$. Çünkü durma noktasında toplama yapılmaz.
- Çözümü kolaylaştırmak için $n = 2^k$ kabul edilir (smoothness rule):

$$A(2^k) = A(2^{k-1}) + 1 = A(2^{k-2}) + 2 = \cdots = A(2^{k-i}) + i$$

- $i = k$ alınır: $A(2^k) = A(1) + k = k$. $n = 2^k$ olduğu için $k = \log_2 n$. Sonuç $A(n) \in \Theta(\log n)$.

**Ders sonrası notlar**

- **Yinelemeli çözüm ve yığın:** Bir öğrenci "yinelemeli çözümler yığın (stack) değil heap kullanır" anlamında bir şey söyledi. Bu doğru değil. Yinelemeli çözümde de yerel değişkenler yığında durur. Fark, özyinelemede her çağrı için yığında yeni bir çerçeve (stack frame) açılmasıdır. Heap yalnız dinamik bellek ayrıldığında kullanılır.
- **UniqueElements iç toplamı:** Derste bir öğrenci iç toplamı "$n - i - 2$" diye söyledi. Doğrusu $\sum_{j=i+1}^{n-1} 1 = (n-1) - (i+1) + 1 = n - 1 - i$.
- **Harmonik toplam:** Altyazıda bu formül anlaşılmıyor. Doğrusu $\sum_{i=1}^{n} \frac{1}{i} \approx \ln n + \gamma$, $\gamma \approx 0{,}5772$ (Euler sabiti). $n = 1000$ için toplam $\approx 7{,}4855$, yaklaşık değer $\approx 7{,}4850$ (Python ile kontrol edildi).
- $\sum i\, 2^i$ **formülü:** Doğrusu $(n-1)2^{n+1} + 2$. $n = 10$ için iki taraf da 18434 verir (Python ile kontrol edildi).
- **BinRec durma koşulu:** Bir öğrenci önce "`n = 0` ise 1 döndür" önerdi. Bu durumda `BinRec(1)` çağrısı `BinRec(0)`'ı çağırır ve 1 fazla sayar. Hocanın son hali (`n = 1` ise 1 döndür) doğrudur. Python ile 1, 7, 8, 29 için doğru basamak sayısı (1, 3, 4, 5) elde edildi.
  - Algoritma $n \geq 1$ için tanımlıdır. `BinRec(0)` sonsuz özyinelemeye girer. Bu yüzden girdi pozitif kabul edilir.
- $A(n)$ **ile basamak sayısı farkı:** $A(n)$ toplama sayısıdır ve $\lfloor \log_2 n \rfloor$'dir. Algoritmanın döndürdüğü basamak sayısı ise $\lfloor \log_2 n \rfloor + 1$'dir. İkisi aynı şey değildir.
- **Toplamada temel işlem:** Bir öğrenci "iki toplama olsaydı $+2$ mi yazardık?" diye sordu. Hoca genelde bir işlem seçildiğini söyledi. Aslında her çağrıda $c$ sabit sayıda işlem yapılıyorsa $+c$ yazılır. Sonuç yine aynı büyüme derecesini verir: $\Theta(n)$ ya da $\Theta(\log n)$.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.